

paint

简单观察可以发现，假设一个颜色最后出现的最左边的一个是 l ，最右边是 r ，则改颜色的刷子粉刷 $[l, r]$ 是不劣的。

接着发现如果存在合法的刷皮鞋序列，则按左端点排序后的序列一定是其中一个。那么对于任意的 x ，它的颜色一定是满足 $l_c \leq x \leq r_c$ 且 l_c 最大的那一个。这可以使用栈从左往右扫来判断。

时间复杂度 $O(n)$ 。

swap

首先建一个 n 个点的完全二叉树，每次操作相当于去看是否交换 k 与其父亲节点。

考虑节点 i 先后与左右儿子交换，有 4 种可能：

1. 不变。
2. i 与左儿子交换。
3. i 与右儿子交换。
4. i 变成其右儿子，左儿子变为 i ，右儿子变为左儿子。

容易想到去贪心，但右儿子最小时不知道应该是 3 还是 4。

进一步去分析可以发现一个点的取值可能是：

- 某个儿子。
- 包含自己在内的某个祖先。
- 某个祖先 u 的兄弟节点 v （额外要求 u 是其父亲的右儿子）。

接下来有两种解决方法：

- 贪心

然后按照 $1 \sim n$ 的顺序去贪心，每次去看这个点能取到的最小值（只有 $O(\log n)$ ）种可能，然后将节点分为三类：

1. 取值不来自于儿子。
2. 取值来自于左儿子。
3. 取值来自于右儿子。

对于第一类情况，其祖先节点不可能换到后代节点上。其左右子树也不能进行交换。

对于第二类情况，其祖先节点有且仅有一个能换到左儿子上，且祖先和左子树不会与右子树交换。

对于第三类情况，其祖先节点有一个可以换到子树内，左儿子也能换到右子树内。

依此进行贪心即可。

时间复杂度 $O(n \log n)$ 。

- 动态规划

令 $f(x, i)$ 表示当 $x = i$ 时, i 最后在哪个位置。转移时遇见右儿子最小的情况时, 不妨假设 $i < a_{ls}$, 此时我们希望将 i 排的靠前, 所以只需判断 $f(ls, i)$ 和 $f(rs, i)$ 哪个更小即可。

由于每个点只会有 $O(\log n)$ 种取值, 所以状态数是 $O(n \log n)$ 的。时间复杂度根据实现的不同可以做到 $O(n \log n)$ 或 $O(n \log^2 n)$ 。

triangle

假设枚举了较小的两条边 $a < b$, 那么面积关于 c 是先递增再递减的, 所以最小值一定取到 c 最小或者 c 最大。

c 最小的情况中, b, c 一定是相邻的两个数, 可以枚举, 固定 b, c 后面面积关于 a 单调递增, 所以只需要找到大于 $c - b$ 的最小的 a 更新答案。

c 最大的情况中, 我们枚举 $a + b = s$, c 一定是小于 s 的最大的数。固定 s 和 c 之后面积关于 a 单调递增, 所以只需要找到最小的 a 使得 a 和 $s - a$ 同时存在, 这可以拿 bitset 移位做 and 来求。

时间复杂度 $O(n + c^2/w)$ 。

sort

设一个数 a_l 后面第一个大于它的数为 a_{r+1} , 则称 $l \sim r$ 为一段。

注意到如果一段数同时在前一半或者后一半, 则其在新序列中仍是一段。所以每次操作本质上是:

- 将跨越中点的那一段分成若干段, 然后将所有段按照开头的元素从小往大排序。

对于求分开之后的段, 可以直接暴力, 均摊 $O(n)$ 。

对于每个数, 维护以其开头的段的长度 (如果不为开头则为 0), 则对于每个询问可以通过线段树二分/树状数组二分来判断询问的第 x 个数在哪一段中, 然后根据原序列可以轻易求出第 x 个数是哪个数。

同时注意到操作至多 n 轮后序列将不再改变, 这是因为如果没有段被分开则序列不会改变, 而段最多分裂 n 次。

时间复杂度 $O(n \log n)$ 。